

Software Specification: PDF-to-Quiz Application

1. Product Overview

The **PDF-to-Quiz App** is a web-based learning tool that allows users to upload educational PDF documents. The system extracts the text from the document and uses the Google Gemini API to automatically generate a multiple-choice quiz based on the content. Users can take the quiz interactively and receive a final score.

2. Target Tech Stack

- **Frontend:** React (Next.js App Router recommended) with Tailwind CSS for styling.
- **Backend:** Next.js API Routes (Serverless functions) or Node.js/Express.
- **PDF Parsing:** pdf-parse (Node.js) or pdfjs-dist .
- **AI Engine:** Google Gemini API (gemini-2.5-flash model).

3. Core Features & User Flow

1. **File Upload:** A drag-and-drop interface for users to upload a single .pdf file.
2. **Loading State:** A visual indicator (spinner/progress bar) while the text is extracted and the AI generates the quiz.
3. **Quiz Interface:**
 - Displays one question at a time or all in a list.
 - Interactive radio buttons or clickable cards for multiple-choice options.
 - State management to track user selections.
4. **Results Screen:**
 - Displays the final calculated score (e.g., "4/5 Correct").
 - Highlights the correct vs. incorrect answers for the user to review.

4. System Architecture & Data Flow

1. **Client:** User uploads document.pdf .
2. **Server:** Backend receives the file and runs the PDF parser to extract raw text.
3. **Server -> Gemini:** Backend constructs a prompt combining the extracted text and strict instructions to return a JSON array.
4. **Gemini -> Server:** Gemini processes the text and returns a structured JSON payload containing the quiz.
5. **Server -> Client:** Backend relays the parsed JSON to the frontend.
6. **Client:** React state (useState) mounts the quiz data and begins the interactive session.

5. Gemini API Integration Details

- **Model Selection:** gemini-2.5-flash (Ideal for high speed, low cost, and massive context windows needed for long PDFs).
- **Feature Requirement: Structured Outputs (JSON Mode).** The API call must use `responseMimeType: "application/json"` and ideally a `responseSchema` to guarantee the LLM never returns markdown text, only raw data.

5.1 Required JSON Data Model

The Gemini prompt must mandate the following output structure:

```
[
  {
    "question": "What is the main function of the mitochondria?",
    "options": [
      "Protein synthesis",
      "Cell division",
      "Energy production",
      "Waste removal"
    ],
    "correctAnswer": "Energy production",
    "explanation": "The text states that the mitochondria is the powerhouse of"
  }
]
```



6. Error Handling & Edge Cases

- **PDF Too Large / Too Many Pages:** Implement a file size limit (e.g., 10MB) on the frontend.
- **Unreadable PDFs:** Scanned PDFs without OCR (Optical Character Recognition) will return empty text. The backend must detect empty extractions and return an error: *"Could not extract text. Please ensure the PDF is a text document, not an image."*
- **Gemini Rate Limits:** The free tier allows 15 Requests Per Minute (RPM). The backend should catch 429 Too Many Requests errors and display a friendly message to the user to try again in a minute.
- **Hallucinations:** Include an "explanation" key in the AI data model so users can understand *why* an answer is correct based on the text.